Knowledge-Augmented Log Anomaly Detection with Large Language Models

Yongliang Tao
Chongqing University
Chongqing, China
yongliangtao@stu.cqu.edu.cn

Hongyu Zhang*
Chongqing University
Chongqing, China
hyzhang@cqu.edu.cn

Van-Hoang Le
The University of Newcastle, Australia
Newcastle, Australia
levanhoang.psa@gmail.com

Yi Xiao
Chongqing University
Chongqing, China
xiaoyi@stu.cqu.edu.cn

Abstract

Log anomaly detection is critical for maintaining system reliability, yet existing large language model (LLM)-based methods suffer from limited accuracy, high computational costs, and poor explainability. In this paper, we introduce LogPipe, a novel framework that enhances LLM-based log anomaly detection by integrating a dynamic knowledge base. LogPipe constructs a knowledge base using discrete and semantic log patterns, augmented by dynamic patterns that are generated by a sentiment dictionary and frequent pattern mining. During inference, log sequences are matched against the knowledge base to provide specific guidance to the LLM, improving detection accuracy and generating detailed explanations. A continuous update mechanism ensures that the knowledge base remains relevant while minimizing redundant LLM queries, significantly reducing inference costs. Evaluated on eight public datasets, LogPipe achieves an average F1 score of 0.975, outperforming state-of-the-art models, with reduced token consumption. Additionally, LogPipe excels in fault localization, which enhances the explainability of detected anomalies.

CCS Concepts

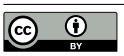
• **Software and its engineering → AI for Software Engineering**.

Keywords

Log Anomaly Detection, Large Language Models, Knowledge Base

ACM Reference Format:
Yongliang Tao, Hongyu Zhang, Van-Hoang Le, and Yi Xiao. 2026. Knowledge-Augmented Log Anomaly Detection with Large Language Models. In *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE '26), April 12–18, 2026, Rio de Janeiro, Brazil.* ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/3744916.3773248

*Corresponding author.



ICSE '26, Rio de Janeiro, Brazil

ACM ISBN 979-8-4007-2025-3/26/04
https://doi.org/10.1145/3744916.3773248

1 Introduction

Log anomaly detection [4, 20, 25, 42, 47, 49] is important for system reliability. In complex networked environments, promptly identifying anomalies based on log data is crucial for fault diagnosis and prevention. Timely detection mitigates cascading failures and ensures reliable operations [13, 24, 31, 48, 50]. In contrast, neglecting anomalies can lead to system crashes [2, 5, 10, 34], causing significant losses.

Traditionally, anomaly detection relied on predefined rules. However, increasing system complexity and log volume pushed the field toward data-driven machine learning methods such as PCA [42], SVM [24], and LogCluster [25]. Recently, deep learning has advanced the field with LSTM-based models [7, 23, 47] that identify anomalies through deviations in event sequences. Furthermore, pre-trained models like BERT [6] have also been adapted, leveraging masked token prediction probabilities to classify anomalies. Although deep learning methods [20, 39, 40, 43, 45, 47] have improved detection performance, they typically lack interpretability and require complex model design.

More recently, LLM-based methods [32, 46] have emerged for log anomaly detection, as LLMs are capable of directly utilizing the semantic content of log messages to detect anomalies and produce human-readable explanations. This provides better interpretability than traditional deep learning approaches. For example, RAGLog [35] retrieves the most similar normal log sequences and uses them as additional context for the LLM to compare against the target log. LogPrompt [29] relies on handcrafted prompts that mark logs containing certain keywords (e.g., "error") as anomalies. However, these methods have not yet achieved satisfactory anomaly detection performance (e.g., LogPrompt reports F1 scores below 0.5 on the BGL and Spirit datasets [28, 29]). Moreover, limited detection effectiveness often suggests that the generated explanations may also be less informative. In addition, current LLM-based methods require sending each log sequence to an LLM for evaluation, which results in high computational costs and hinders practical applications.

We believe that the limited effectiveness of LLM-based anomaly detection methods stems from their inability to provide LLMs with specific guidance. For example, LogPrompt mainly relies on simple and incomplete prompts—such as marking any log containing the word "error" as anomalous—to guide the LLM. However, in many real systems, normal logs may also contain such negative words, which leads to misclassifications. Existing RAG-based methods,

such as RAGLog, attempt to provide guidance by retrieving normal log sequences that are semantically similar in the embedding space to the log sequence to be detected, and guiding the LLM to compare them. Yet this guidance remains relatively coarse, as it is mainly based on similarity retrieval and fails to highlight the discriminative patterns that distinguish anomalies from normal log sequences. Moreover, the retrieved context can become relatively long, which increases inference cost and may even confuse the LLM [22, 26], ultimately reducing the detection effectiveness.

To provide LLMs with specific guidance in log anomaly detection, we propose **LogPipe**, a framework that integrates a knowledge base with an LLM for knowledge-enhanced anomaly detection. We construct a knowledge base that systematically stores discriminative patterns mined from historical log data, which are helpful for anomaly detection. For each type of pattern, we further provide specific guidance to the LLM. Compared to directly providing the LLM with lengthy context or simple keyword prompts, this approach offers more specific guidance, which we expect to help the LLM make better classification decisions.

Specifically, we design a multi-faceted mining strategy to extract discriminative patterns. We extract discrete anomalous event patterns, defined as log events that occur only in anomalous sequences, using two methods: the traditional set difference approach and a novel semantic mining method that leverages the In-Context Learning capability of LLMs. Meanwhile, we apply set-based deduplication to extract discrete normal event patterns, where each individual log event is inherently considered normal. Both types of events are treated as discrete patterns. To further capture anomalies that are semantically similar, we observe that many log sequences, while not identical in content, exhibit high similarity in their embedding representations. We leverage this property to construct semantic patterns using a vector database of log sequence embeddings. Additionally, to make the knowledge base more comprehensive, we discover dynamic patterns through the combination of a log event sentiment dictionary that stores the sentiment (negative, neutral, positive) of each log event and frequent pattern mining. These patterns are stored into a knowledge base.

During the inference phase, each log sequence retrieves and matches patterns from the knowledge base. These patterns are then paired with corresponding extra information as guidance, which guides the LLM to judge the status of the log sequence and provides corresponding explanations. This process enhances both the detection accuracy and the explainability of the LLM's output. Furthermore, we employ two key strategies. First, to reduce inference cost, we cache the LLM's conclusions as triplets (pattern–status–explanation). This avoids redundant judgments on log sequences with similar patterns. Second, to keep the knowledge base relevant and comprehensive over time, we continuously extract new patterns from the logs and use them to update the knowledge base.

We evaluated the anomaly detection performance and token consumption of LogPipe on eight public datasets from LogHub [51] and EvLog [16]. Experimental results demonstrate that our proposed LogPipe achieves an average F1 score of 0.975, outperforming state-of-the-art models. Moreover, LogPipe achieves a 5.711× reduction in LLM cost compared to direct LLM-based detection without using the knowledge base's cache; specifically, its token consumption accounts for only 12.03% of RAGLog and 18.96% of LogPrompt, which highlights its significantly lower resource requirements and improved practical applicability. Additionally, LogPipe possesses the capability of locating anomalous logs, which enables specific guidance for LLMs, thereby improving the quality of generated explanations.

Our major contributions are as follows:

- We propose LogPipe, a novel framework that enhances LLM-based log anomaly detection with a dynamic knowledge base, improving accuracy, efficiency, and explainability compared to existing methods.
- We construct a knowledge base using various patterns, which capture normal and abnormal log behaviors and guide the LLM to provide accurate detection and detailed explanations.
- We perform extensive experiments to evaluate the proposed LogPipe, and the results confirm the effectiveness and efficiency of the proposed approach.

2 Background and Motivation

2.1 Log Anomaly Detection

Log-based anomaly detection aims to identify abnormal behaviors by analyzing system-generated log data. Existing methods can be mainly categorized into three types: traditional machine learning methods, deep learning-based approaches, and more recently, LLM-based methods.

Traditional machine learning methods typically rely on count vectors combined with shallow classifiers such as Support Vector Machine (SVM) [24], Decision Tree [4], KNN [44], and Logistic Regression [3]. These approaches are simple, effective in low-dimensional settings. However, they heavily depend on feature engineering and often fail to capture complex sequential dependencies in log data, limiting their generalization ability across diverse systems. Deep learning techniques like CNN [31] are used to classify a log sequence into normal or abnormal. Models such as LogRobust [47] adopt LSTM to detect anomalies, while Transformer-based architectures like LogFormer [14], NeuralLog [19], and PreLog [21] are leveraged to better capture semantic relationships in log sequences. While effective in performance, deep learning approaches generally offer limited interpretability due to their black-box nature.

More recently, LLM-based methods have emerged, leveraging pre-trained models for log semantics. RAGLog [35] enhances contextual understanding via retrieval-augmented generation, and LogPrompt [29] applies prompt engineering to detect anomalies using task-specific instructions. LLM-based methods benefit from rich semantic knowledge encoded in large-scale language models and show potential in few-shot or zero-shot settings. However, their current anomaly detection performance is still suboptimal because they cannot provide more specific guidance to the LLM. We describe the limitations of these methods in the next subsection.

2.2 Limitations of LLM-Based Log Anomaly Detectors

To better understand the challenges faced by recent LLM-based log anomaly detection methods [29, 35], we analyze their limitations in terms of detection performance and computational cost. We

note that there exist several LLM-based log anomaly detection approaches [1, 12, 28] that rely on fine-tuning model parameters. However, these methods typically require retraining for each new dataset and entail significant computational cost, which makes them less practical to reproduce and compare fairly in our experimental setting. Therefore, we focus our study on representative prompt-based and retrieval-augmented methods, such as LogPrompt and RAGLog, which are more relevant to our approach as they apply LLMs **without modifying model parameters**.

To ensure a fair evaluation, we reproduced the results of RAGLog and LogPrompt by following the parameter settings described in the original papers and running their publicly available code. We conducted experiments on two widely used public datasets, Spirit and BGL [51]. For both datasets, we split log sequences into fixed-length windows of 100 events. Following common practice, we used the most recent 20% of each dataset, based on the timestamp order, as the test set.

Experimental results show that RAGLog and LogPrompt suffer from three primary limitations that hinder their real-world applicability: **limited anomaly detection accuracy, high inference costs, and limited interpretability**. As detailed in Table 1, both methods exhibit these drawbacks. For example, RAGLog's consumption of 49 million tokens on the BGL dataset highlights the significant token expenditure that makes it impractical for large-scale deployment. Furthermore, the insufficient accuracy of anomaly detection (F1 score of 0.549) influences the trustworthiness of anomaly explanations.

Table 1: Comparison of LLM-based Log Anomaly Detectors

Model	Precision	Recall	F1	Token Usage	Dataset
RAGLog	0.381	0.988	0.549	49,424,963	BGL
RAGLog	0.251	0.642	0.359	69,358,290	Spirit
LogPrompt	0.220	0.311	0.258	29,380,434	BGL
LogPrompt	0.215	0.229	0.224	35,593,763	Spirit

RAGLog's limited anomaly detection accuracy primarily stems from its insufficient extra information, which prevents it from providing specific guidance to the LLM. This mechanism merely provides the most similar normal log sequences to the LLM, **yet it fails to effectively leverage the deep, truly discriminative patterns within log sequences.** Consequently, this prevents significant improvements in the LLMs' anomaly detection accuracy. Furthermore, feeding these most similar log sequences as input not only increases the LLM's inference cost (e.g., token consumption) but can also confuse the LLM due to the overly long context, ultimately reducing detection accuracy.

Given these limitations, we contend that analyzing discriminative patterns from log sequences, capable of distinguishing normal from anomalous behaviors, is crucial for enhancing LLMs' log anomaly detection capabilities.

3 Approach

3.1 Overview

Our proposed framework consists of three stages, as illustrated in Figure 1. First, we construct a knowledge base of patterns through mining. When a new log sequence arrives, it first retrieves patterns from the knowledge base. If the retrieved pattern has a corresponding status, the system directly returns the associated status and explanation. Otherwise, the pattern is paired with its corresponding extra information to knowledge-augment the LLM, which then determines the log status and provides an explanation. Finally, in the third stage, the retrieved pattern, along with its inferred status and explanation, is added to the knowledge base as a pattern-status-explanation triplet to support continuous updates.

More specifically, in the **first stage**, we construct a comprehensive *knowledge base* by mining discriminative patterns from historical log data. Specifically, the knowledge base integrates three complementary types of patterns:

- Discrete patterns: including discrete normal event patterns that represent common normal log events, and discrete anomalous event patterns that represent log events that occur only in anomalous sequences. They serve as static and highly discriminative patterns for anomaly detection.
- Semantic patterns: a vector database that stores embeddings of historical log sequences. When a new log sequence arrives, it is encoded into an embedding and compared against the database to retrieve semantically similar historical sequences, producing an anomaly score to help detect potential anomalies.
- Dynamic patterns: high-risk infrequent patterns and sub-threshold infrequent patterns dynamically identified by combining frequent pattern mining with a log event sentiment dictionary. The system counts the number of candidate high-risk patterns in a new log sequence and compares it to a predefined threshold T (optimized on a validation set): if it exceeds the threshold T, the sequence is classified as high-risk infrequent patterns; otherwise, it is considered sub-threshold infrequent patterns.

In the **second stage**, for each new log sequence, we retrieve matching patterns from the knowledge base. Each matched pattern is associated with extra information that serves as contextual guidance. This extra information, combined with the original log sequence, forms a prompt [11, 27, 36] that enables the LLM to judge the status of the log sequence and generate a human-readable explanation. By leveraging these complementary patterns and prompt-based extra information, the framework helps the LLM reduce false positives and improve detection accuracy.

The **third stage** focuses on continuous knowledge base update. This serves two purposes: (1) caching LLM-judged triplets (pattern–status–explanation) to avoid redundant LLM queries for similar patterns, and (2) extracting new patterns and adding them to the knowledge base.

3.2 Constructing Knowledge Base

The knowledge base consists of three parts. The first part is discrete patterns, which include discrete normal event patterns and discrete anomalous event patterns. The second part is semantic patterns, where a vector database stores embeddings of historical log sequences. The third part is dynamic patterns, generated by combining frequent patterns with a log event sentiment dictionary.

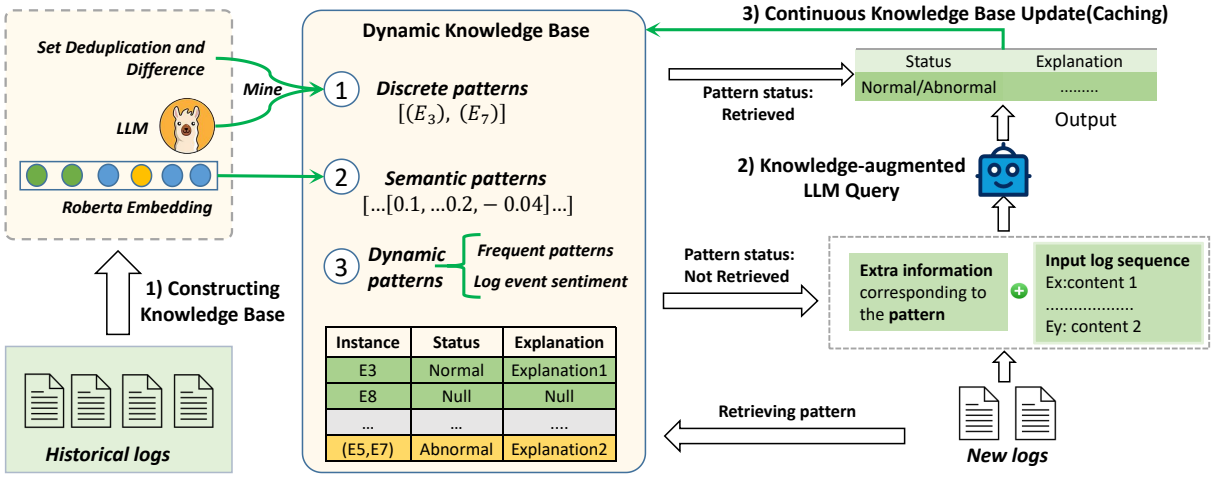


Figure 1: An Overview of LogPipe.

1) Mining Discrete Patterns. **Discrete anomalous event patterns** are discriminative patterns for log anomaly detection. We employ set difference operations and LLM In-Context Learning to extract these for knowledge base construction. We randomly sample normal and abnormal log sequences from the training set, storing them in set A (normal) and set B (abnormal), respectively. **Set deduplication** on the collected log event IDs from sequences in set A yields the **discrete normal event patterns**, which are stored in the normal event set (e.g., $\{E_1, E_2, ..., E_K, ...\}$).

For an abnormal sequence S_{ab} in set B, let its abstracted events be represented as a sequence $(E_x, E_y, \ldots, E_z)$. We compute the **set difference** $D_S = \{E_x, E_y, \ldots, E_z\} \backslash$ normal event set. If D_S contains only a single log event or repeated occurrences of the same log event ($|Unique(D_S)| = 1$), that event $e \in Unique(D_S)$ is a **discrete anomalous event pattern**, and is added to the **anomalous event set**.

While this structural method captures **discrete anomalous event patterns** efficiently, it neglects semantic context in logs. To mine more discrete anomalous event patterns, we utilize LLMs for further analysis. Let $E_{flagged\_anomalous}$ be the set of all discrete anomalous event patterns identified by the structural method. We consider a set of complex difference sets $D_{complex} = \{D_S \mid |Unique(D_S)| > 1\}$. For each $D'_S \in D_{complex}$, we first check if $Unique(D'_S) \cap E_{flagged\_anomalous} = \emptyset$. If this condition holds (i.e. the complex difference set contains no already flagged discrete anomalous event patterns), then we input its log template IDs and corresponding log messages into an LLM. Leveraging its **In-Context Learning (ICL)** capabilities, the LLM can identify the anomalous events of high impact. These identified events are newly found discrete anomalous event patterns.

As shown in Figure 2, we input the log template IDs along with their corresponding log messages into the LLM. Leveraging its powerful In-Context Learning capabilities, the LLM is guided to semantically determine which log indicates a more critical failure. The LLM identifies E106 as more significant, as it provides a detailed diagnosis of an aborted journal and reveals the root cause of the filesystem error. The identified log template ID is new found discrete

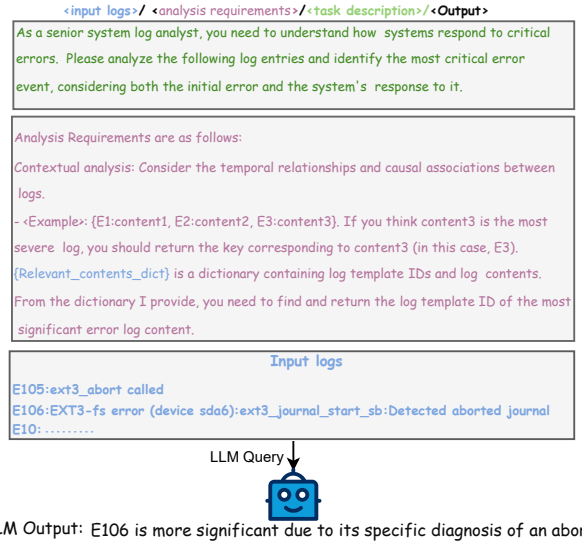


Figure 2: Using LLM to mine discrete anomalous event patterns with ICL

anomalous event pattern. Due to space limits, we show only part of the prompts. Details can be found at our project repository.

In this section, we mine discrete patterns, specifically including discrete normal event patterns and discrete anomalous event patterns. At inference time, when new log sequences match these patterns, we pair them with their corresponding extra information to guide LLM-based anomaly detection. Extra information for discrete normal event patterns establishes expected behavior, reducing false positives. Extra information for discrete anomalous event patterns highlights potential anomalies, improving detection accuracy.

2) Mining Semantic Patterns. Beyond the discrete patterns, the raw log sequences themselves contain rich semantic information. To further leverage the semantic features of normal (set A) and

abnormal (set B) log sequences, we construct a vector database as semantic patterns. We employ RoBERTa [6] to encode these log sequences into high-dimensional vector representations. This vectorization enables us to capture the nuanced semantic relationships between different log sequences, as log sequences representing similar system states tend to exhibit close semantic proximity.

For a new log sequence, we first encode it with the same RoBERTa model. Its top-k (100 in our experiments) most semantically similar historical sequences are then retrieved from the vector database using cosine similarity. The anomaly score is defined as the number of anomalous sequences within these top-k retrieved sequences. If this score exceeds a predetermined threshold (optimized on a validation set), the sequence is considered a potential anomaly. Its anomaly score acts as extra information, enabling the LLM to refine the determination of the log status and provide a specific explanation for its judgment.

3) Mining Dynamic Patterns. The first two parts of our knowledge base store static patterns. However, log systems continuously generate new data, making it impossible for the knowledge base to provide similar patterns for every log sequence. To address this challenge, we integrate frequent pattern mining with sentiment analysis to dynamically discover new patterns and generate the corresponding extra information. Some researchers [37] show that log events with positive or neutral sentiments typically indicate normal logs. Leveraging this crucial insight, we assign sentiment labels (e.g., negative, neutral, positive) to log events. This offers two primary advantages: 1) it enables us to filter out newly generated normal log sequences—specifically, those composed solely of neutral or positive events. 2) we remove neutral and positive sentiment log events from log sequences, retaining only negative events within those sequences, which significantly sharpens our focus on potential anomalous logs.

The process begins with the construction of a **log event sentiment dictionary**. We group logs by template ID and assign each ID a sentiment label by feeding a representative log statement associated with each template ID into a pre-trained RoBERTa sentiment classifier [30]. This yields a log event sentiment dictionary (E1:Negative, E2:Neutral). To establish a statistical baseline for normal behavior, we extract frequent patterns (N) from normal log sequences. We apply a sliding window (with a maximum length of 3) over normal sequences in the training set and retain fragments whose frequency exceeds a predefined support threshold (2 occurrences in our experiments). These fragments form the **frequent patterns**, serving as a statistical baseline for normal behavior.

We design an algorithm (Algorithm 1) to discover dynamic patterns, which improves the detection of previously unseen log sequences. The details of Algorithm 1 are presented as follows: given a log sequence, we first deduplicate it and remove all known normal templates (from the normal event set) and then perform sentiment analysis on the remaining entries. If all exhibit neutral or positive sentiment, this sequence can be classified as a sub-threshold infrequent pattern. Otherwise, only the negatively labeled logs are retained as suspicious segments. We then mine frequent patterns from these segments and compare them with N. Any newly discovered pattern not included in N is treated as a **candidate high-risk pattern**, and a counter is incremented accordingly. Once this counter exceeds a preset threshold T (optimized on a validation set), the sequence is classified as containing **high-risk infrequent patterns**, implying it is anomalous; otherwise, it is considered to exhibit **sub-threshold infrequent patterns**, implying that it is normal.

Algorithm 1 Combine Sentiment With Frequency Pattern

Input: S (Input log sequence), N (Frequent patterns), T (Anomaly threshold), NS (Normal event set)
Output: Pattern category

$S' \leftarrow Set(S) - NS$ {Remove normal Log Templates}
if $\forall s \in S'$, sentiment$(s) \in$ {neutral, positive} **then**
  return Sub-threshold infrequent patterns
end if
$C \leftarrow 0$ {Initialize count}
Remove $s \in S'$ with neutral/positive sentiment
for $l \leftarrow 1$ to MAX_WINDOW_SIZE **do**
  for $i \leftarrow 1$ to $|S'| - l + 1$ **do**
    $snippet \leftarrow (S'[i], ..., S'[i + l - 1])$
    if $snippet \notin N$ **then**
      $C \leftarrow C + 1$
    end if
  end for
end for
if $C \geq T$ **then**
  return High-risk infrequent patterns
else
  return Sub-threshold infrequent patterns
end if

By combining the log event sentiment dictionary with existing frequent patterns, we can dynamically generate new patterns (high-risk and sub-threshold infrequent patterns). Their corresponding extra information helps the LLM perform anomaly detection.

3.3 Knowledge-Augmented LLM Query

In previous sections, we introduced the composition of the knowledge base. The knowledge base stores three types of patterns: discrete patterns (e.g., E1, E2, E3), dynamic patterns (e.g., (E5, E7)), and semantic patterns of log sequences stored in a vector database. The retrieval process is relatively straightforward. Specifically, the vector database retrieves semantic patterns through cosine similarity matching, while discrete patterns and dynamic patterns can be retrieved via simple string matching.

When a new log sequence matches a pattern stored in the knowledge base, we pair it with the corresponding extra information. As shown in Table 2, the extra information consists of textual descriptions highly relevant to the matched patterns, which are provided to the LLM to offer specific guidance. Together with the original log sequence, this extra information helps the LLM refine its judgment and produce more informative explanations. Figure 3 illustrates a complete example: the log sequence first retrieves that E10 is a discrete anomalous event pattern from the knowledge base; then, it is paired with corresponding extra information, and both are submitted to the LLM. Finally, the LLM outputs the anomaly judgment ("abnormal") along with an explanation.

Table 2: Examples of Corresponding Extra Information for Different Patterns

Pattern	Pattern's Source	Corresponding Extra information
Discrete anomalous event patterns	Anomalous event set	This log sequence contains unique anomalous log events, Ex:content is a unique anomalous log event...
Discrete normal event patterns	Normal event set	Perhaps this log sequence contains many keywords (e.g., "error", "fail"), but please do not judge normality or abnormality based on keywords... You can trust that every single log is normal.
Semantic patterns	Vector Database	The similarity between this log statement and the anomalous logs stored in the database has reached {score}%, and reaches an abnormal score...
High-risk infrequent patterns (Dynamic patterns)	Frequent patterns and a log event sentiment dict	In this log sequence, there are infrequent segments, their count reached the anomaly threshold for infrequent items, which is considered quite reliable...
Sub-threshold infrequent patterns (Dynamic patterns)	Frequent patterns and a log event sentiment dict	In this log sequence, there are some infrequent segments, but their count has not yet reached the anomaly threshold for infrequent items...

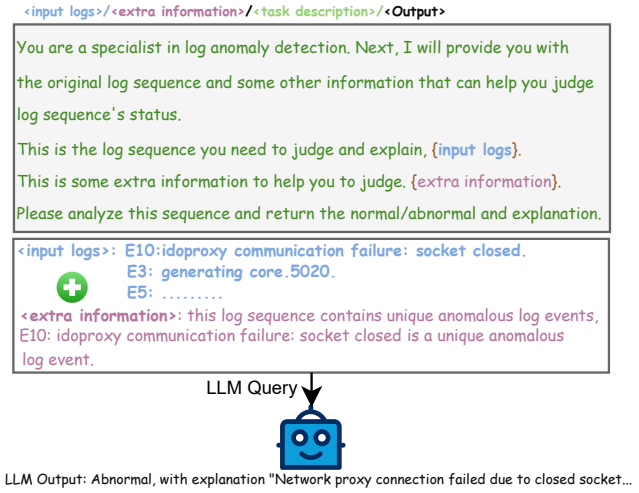


Figure 3: Giving extra information to help the LLM with anomaly detection and explanation generation

The extra information associated with these patterns greatly aids the LLM's judgment. Specifically, the extra information corresponding to discrete anomalous event patterns, semantic patterns, and high-risk infrequent patterns provides the LLM with insights about anomalies. This mitigates common LLM issues like hallucinations [8, 38], making its anomaly detection judgments and explanations more reliable. In contrast, the extra information corresponding to discrete normal event patterns and sub-threshold infrequent patterns can provide the LLM with insights about expected normal behaviors in log sequences, enabling it to reduce false positives.

3.4 Continuous Knowledge Base Update

After receiving log status and explanation produced by the LLM, our framework initiates a continuous knowledge base update process. This process primarily serves two purposes: (1) caching triplets (pattern–status–explanation) to avoid redundant LLM queries for similar patterns in the future; and (2) extracting new patterns.

The first aspect involves caching LLM judgments. When a new log sequence is processed and its status judgment and explanation are provided by the LLM, we store "pattern–status-explanation" triplet for that specific pattern instance. For example, as shown in Table 2, suppose a log sequence contains a specific discrete anomalous event instance (E10). If the LLM judges this sequence as abnormal and provides an explanation, we update the knowledge base by adding a triplet such as "(E10)–abnormal–explanation". This approach also applies to discrete normal event patterns and dynamic patterns. By storing their pattern–status–explanation triplets, the knowledge base becomes increasingly comprehensive. When a similar pattern appears again, the system can directly retrieve the stored status and explanation instead of querying the LLM, thus significantly reducing computational costs.

Beyond caching, another key function of our framework is to extract and integrate new pattern instances from LLM-confirmed anomalies, enabling the knowledge base to learn dynamically and evolve. Semantic patterns retrieved from the vector database can also lead to the addition of new discrete anomalous log events, but only under some conditions. Specifically, when a log sequence is identified as semantically anomalous and subsequently confirmed as abnormal by the LLM, if the entire sequence is composed of repetitions of the same log template ID, we conduct an additional LLM verification. This check aims to confirm that this single log template itself inherently implies an anomaly. Once this template is also confirmed as anomalous, its log template ID is stored in the knowledge base as a new discrete anomalous event pattern.

Table 3: Examples of Pattern–Status–Explanation Triplets (A: Abnormal, Ex: Explanation).

Pattern	Status	Ex	Type
E18	A	...	Discrete anomalous event pattern
(E8,E9)	A	...	High-risk infrequent pattern
E38	A	...	Discrete anomalous event pattern

Table 3 shows several pattern–status–explanation triplets. Each row corresponds to a single pattern instance, showing its identifier,

Table 4: Comparison of Log Anomaly Detection Models on Eight Log Datasets

Category	Model	Metric	BGL	Spirit	Thunderbird	HDFS	Hadoop2	Hadoop3	Spark2	Spark3	Average
ML-based	Decision Tree	Precision	**1.000**	**1.000**	**1.000**	**1.000**	**1.000**	**1.000**	**1.000**	**1.000**	**1.000**
		Recall	0.370	0.367	**1.000**	0.998	**0.996**	0.995	0.950	0.886	0.820
		F1	0.540	0.537	**1.000**	**0.998**	**0.998**	0.996	0.974	0.939	0.872
	SVM	Precision	0.847	**1.000**	0.607	0.959	**1.000**	**1.000**	**1.000**	**1.000**	0.926
		Recall	0.368	0.355	0.739	0.970	**0.996**	0.995	0.950	0.886	0.782
		F1	0.513	0.525	0.667	0.959	**0.998**	0.996	0.974	0.939	0.821
	KNN	Precision	0.935	0.188	0.148	0.997	0.995	0.998	0.950	**1.000**	0.776
		Recall	0.734	0.832	0.957	0.999	0.995	0.995	0.950	0.886	0.918
		F1	0.822	0.307	0.256	**0.998**	0.995	0.995	0.950	0.939	0.782
DL-based	LogRobust	Precision	0.696	0.916	0.923	0.961	**1.000**	**1.000**	**1.000**	0.950	0.930
		Recall	0.968	0.287	0.632	**1.000**	0.988	0.991	0.950	0.886	0.837
		F1	0.810	0.437	0.444	0.980	0.994	0.992	0.974	0.917	0.818
	CNN	Precision	0.698	0.964	**1.000**	0.966	**1.000**	**1.000**	**1.000**	**1.000**	0.953
		Recall	0.965	0.310	0.400	**1.000**	0.977	0.993	0.900	0.886	0.803
		F1	0.810	0.470	0.571	0.982	0.982	0.994	0.947	0.939	0.836
	LogFormer	Precision	0.969	0.956	0.950	0.985	**1.000**	**1.000**	**1.000**	0.950	0.976
		Recall	0.914	0.318	0.826	0.985	0.985	0.987	0.900	0.900	0.851
		F1	0.941	0.472	0.883	0.985	0.993	0.990	0.947	0.931	0.892
	LogAnomaly	Precision	0.191	0.790	0.037	0.893	0.926	0.939	0.931	0.898	0.700
		Recall	0.974	0.864	0.960	0.961	0.994	0.988	0.939	0.947	0.953
		F1	0.320	0.830	0.072	0.966	0.958	0.963	0.935	0.922	0.745
	DeepLog	Precision	0.174	0.530	**1.000**	0.935	0.911	0.925	0.862	0.857	0.774
		Recall	0.998	**1.000**	0.005	0.994	0.981	**1.000**	0.952	0.977	0.863
		F1	0.296	0.693	0.010	0.908	0.941	0.961	0.905	0.914	0.703
	NeuralLog	Precision	0.926	0.964	**1.000**	0.963	**1.000**	**1.000**	**1.000**	**1.000**	0.982
		Recall	0.849	0.308	0.621	**1.000**	0.993	0.995	0.950	0.902	0.827
		F1	0.886	0.467	0.766	0.981	0.996	0.997	0.974	0.948	0.877
LLM-based	RAGLog	Precision	0.381	0.250	0.521	0.319	0.453	0.427	0.129	0.122	0.325
		Recall	**0.988**	0.642	0.826	0.575	0.677	0.691	0.850	0.829	0.759
		F1	0.549	0.359	0.638	0.411	0.543	0.528	0.224	0.213	0.433
	LogPrompt	Precision	0.220	0.215	0.478	0.045	0.232	0.241	0.028	0.053	0.214
		Recall	0.311	0.229	0.652	0.197	0.356	0.345	0.250	0.343	0.335
		F1	0.258	0.224	0.551	0.073	0.281	0.283	0.051	0.092	0.226
	LogPipe	Precision	0.931	0.912	0.971	0.998	0.995	**1.000**	**1.000**	**1.000**	0.976
		Recall	0.971	0.925	**1.000**	0.991	0.997	0.996	**1.000**	**0.943**	**0.977**
		F1	**0.951**	**0.917**	0.980	0.993	0.996	**0.997**	**1.000**	**0.971**	**0.975**

the status determined by the LLM, and the associated explanation. The "Type" column provides the category of each pattern. For example, the first instance (E18) is a discrete anomalous event pattern, with status A (abnormal) and the explanation produced by the LLM. These examples illustrate the structure of the triplets maintained in our continuous knowledge base, which is continuously updated to remain relevant and comprehensive, while minimizing redundant LLM queries and reducing inference costs.

4 Experimental Design

Research Questions. We evaluate our approach by answering the following research questions (RQs):

- RQ1: How effective is LogPipe?
- RQ2: How do different modules contribute to LogPipe?
- RQ3: How does LogPipe perform in terms of explainability?

Datasets. We evaluated LogPipe on eight public datasets [16, 51], covering both session-based (HDFS, Hadoop2, Hadoop3, Spark2, Spark3) and fixed-length block segmentation (BGL, Spirit, Thunderbird). Logs are grouped by their BlockID in session-based datasets. In fixed-length segmentation, logs are ordered by time and grouped into windows of fixed size, with 100 logs per window for BGL and Spirit, and 200 logs per window for Thunderbird. We chose 200L over 100L for Thunderbird to reduce the number of API calls by half, as the larger window size leads to comparable detection performance (F1-score: 0.980 vs. 0.968). Data splitting followed a time-based scheme: 80% for training (with the last 10% used for validation) and 20% for testing. For Hadoop2, Hadoop3, Spark2, and Spark3, we followed the partitioning in [16] and directly used

the provided train/validation/test splits. Within the training set, we retain all normal log sequences and randomly sample 2,000 anomalous sequences. This procedure ensures temporal consistency and prevents future information leakage, providing a fair basis for evaluation. Notably, as observed by Landauer et al. [18], random sampling of normal and anomalous sequences can yield artificially optimistic performance estimates, whereas chronological splitting provides a more realistic evaluation. Our time-aware split addresses this concern while maintaining comparability with prior work.

Baselines. To ensure a comprehensive comparison, we select representative state-of-the-art models from different categories, including SVM [24], KNN [44], and Decision Tree [4] as classic machine learning baselines; NeuralLog [19], LogAnomaly [33], DeepLog [7], LogRobust [47], CNN [31], and LogFormer [14] as representative deep learning methods; as well as RAGLog [35] and LogPrompt [29] as recent LLM-based approaches. RAGLog and LogPrompt have publicly available code and are comparable to our method as they require no fine-tuning. The baseline models are implemented as follows to ensure fairness: the classic machine learning models (SVM, KNN, Decision Tree) are based on open-source implementations [15, 44]; the deep learning models (CNN, NeuralLog, LogAnomaly, DeepLog, LogRobust) are sourced from an open-source log analytics repository [20], and LogFormer is reproduced from its official repository [14]. Furthermore, LLM-based models (RAGLog and LogPrompt) are implemented based on their official repositories [29, 35]. All models are evaluated under the same preprocessing pipeline and data splits.

Metrics. Treating log-based anomaly detection as a binary classification task, we evaluate our approach using three commonly used metrics: Precision, Recall, and F1-score. These metrics are widely adopted in anomaly detection tasks to measure detection accuracy and robustness. Furthermore, to assess the effectiveness of LogPipe in fault localization, we adopt HR@K (i.e., top-k), which measures the model's ability to correctly identify anomalous logs within the top-k candidates.

Implementation. As described in Section 3.2, LogPipe involves two thresholds: the anomaly score threshold (Eq. 3.2) and the dynamic pattern threshold T (Algorithm 1). To identify optimal threshold configurations, we performed grid search on the validation set. Following established practices in log analytics research, we extracted the last 10% of the training data (in chronological order) as the validation set. The grid search systematically evaluated the combinations of thresholds to identify the configuration that maximizes the F1 score on the validation set.

All experiments were conducted on a GPU server equipped with an NVIDIA A800 GPU and CUDA 12.2. The implementation utilized Python 3.12 and PyTorch 2.5.1. In this experiment, we selected GLM4-2.0-Flash [9] as the LLM. Additionally, we set the temperature parameter of the LLM to 0 to reduce randomness. Furthermore, in our experiments, we run each setting independently with five different random seeds and report the average performance to further reduce variance.

5 Experimental Results

5.1 RQ1: How effective is LogPipe?

As shown in Table 4, LogPipe achieves an average F1-score of 0.975 across all datasets, outperforming the strongest deep learning baseline, LogFormer (0.892), by 8.3%. Notably, LogPipe shows much larger gains on the Thunderbird and Spirit datasets, with improvements of 9.7% and 44.5%, respectively.

LogFormer is designed to capture complex log sequence patterns, and performs well with sufficiently labeled anomaly samples. However, for fair comparison, it was trained on only 2,000 labeled anomaly samples in our experiments. Given that the Spirit dataset is anomaly-dominant, this limited number of anomaly labels may not provide sufficient information for LogFormer to learn representative anomaly characteristics, resulting in an F1 score of 0.472. In contrast, LogPipe is built upon LLM, which inherently possesses capability for log understanding and anomaly detection. Building on this foundation, LogPipe further leverages discriminative patterns mined from historical logs to provide structured and specific extra information to the LLM. These include discrete anomalous event patterns, semantic patterns, and high-risk infrequent patterns - all of which help guide the LLM toward more accurate anomaly judgments. As a result, LogPipe achieves an F1 score of 0.917 on the Spirit dataset with limited labeled data.

In contrast, LLM-based methods like LogPrompt and RAGLog do not require supervised training, but introduce other challenges. In LogPrompt, the prompt explicitly instructs the model to label a log as anomalous only when it contains error-related keywords such as "error". This prompt itself is incomplete, as many anomalous logs do not include such keywords, leading to low recall. Moreover, many normal logs may still contain words like "error", which lowers precision. As a result, LogPrompt shows poor overall performance.

RAGLog retrieves the most similar log sequences as additional context for the LLM to reference. However, the log sequences to be detected may not be accurately related to the provided context, even if the similarity is high. Secondly, including these additional sequences significantly increases the input length (especially when each log sequence contains around 100 entries), which can confuse the LLM and impair its reasoning ability. Thirdly, RAGLog overlooks patterns inherent in the logs themselves, merely supplying similar sequences without providing the LLM with specific guidance.

In contrast, LogPipe mines inherent discriminative patterns from the logs themselves to build a knowledge base, and assigns more specific extra information to each pattern to guide the LLM. This provides clearer and more specific guidance to the LLM for anomaly identification. The discrete anomalous event patterns, semantic patterns, and high-risk infrequent patterns provide clearer extra information for the LLM to identify anomalies, improving recall. Meanwhile, the extra information associated with discrete normal event patterns informs the LLM about normal log events, helping reduce false positives. As a result, LogPipe outperforms RAGLog in both precision and recall.

In real-world scenarios, logs are continuously generated at a high rate, posing significant inference burdens and cost challenges for LLM-based anomaly detection. Both LogPrompt and RAGLog require invoking the LLM for every new log sequence, resulting in substantial token consumption and computational overhead. To

Table 5: Token Consumption of Different Methods

Method	Token Count	Ratio
LogPipe (+ Cache)	40,824,996	—
LogPipe (No Cache)	233,174,500	17.50%
RAGLog (No Cache)	339,346,953	12.03%
LogPrompt (No Cache)	215,324,376	18.96%

Ratio = Baseline / Current, where Baseline is LogPipe (+ Cache)

address this issue, LogPipe caches the "pattern–status–explanation" triplets after the LLM judges log patterns, enabling direct reuse of existing conclusions when encountering similar patterns in the future and thus avoiding redundant inference. This mechanism significantly reduces token consumption, achieving a 5.711× reduction in token usage and effectively lowering costs. Table 5 demonstrates the substantial savings enabled by this caching strategy. The token consumption of LogPipe with cached judgment is only 12.03% of that of RAGLog and 18.96% of that of LogPrompt. The token consumption for each dataset is shown in Table 6.

Table 6: Comparison of Token Consumption on Each Dataset

System	RAGLog	LogPrompt	LogPipe
BGL	49,424,963	29,380,434	**761,977**
Spirit	69,358,290	35,593,763	**7,188,151**
TB	78,950,439	41,425,461	**29,905,519**
HDFS	71,051,641	64,167,313	**103,036**
Hadoop2	33,998,000	17,161,302	**859,522**
Hadoop3	26,062,872	21,159,209	**1,159,054**
Spark2	3,814,256	2,326,525	**373,294**
Spark3	6,686,492	4,110,369	**474,443**

Summary. The experimental results demonstrate the effectiveness of LogPipe in anomaly detection across diverse system logs. Moreover, since LogPipe caches pattern-status-explanation triples in the knowledge base, it significantly reduces redundant LLM queries.

5.2 RQ2: How do different modules contribute to LogPipe?

To answer this RQ, we conducted an ablation study on eight log datasets and report the averaged results. The ablation study (Table 7) shows that each part of the knowledge base contributes to the improved detection accuracy. Using only the knowledge base without the LLM (KB only) achieves moderate results (Precision: 0.834, Recall: 0.850, F1: 0.842). This is because the LLM provides additional understanding of log semantics: it can correctly identify anomalous sequences even when the KB mislabels them. Furthermore, the LLM helps discover more discrete anomalous event patterns during both the knowledge base construction and the subsequent extraction of new patterns, enriching the knowledge base. Conversely, a simple LLM without the knowledge base heavily relies on keywords (e.g., "error"), resulting in low precision (0.380) despite high recall (0.946).

Combining KB with LLM significantly improves detection performance. Adding discrete patterns significantly increases precision to 0.793 by allowing the system to recognize known normal and abnormal behaviors. Including semantic patterns further improves recall to 0.968 and raises the F1 score to 0.889, as more semantic matches help catch unseen but similar anomalies. Adding dynamic patterns provides extra information that guides the LLM when no patterns are matched, boosting precision to 0.963 and F1 to 0.965. Finally, enabling continuous updates (Full LogPipe) achieves the best performance (F1=0.975), confirming that each part incrementally improves the model and dynamic updates further enhance accuracy.

Table 7: Ablation Study on LogPipe

Setting	Precision	Recall	F1
Simple LLM (w/o KB)	0.380	0.946	0.542
KB only (w/o LLM)	0.834	0.850	0.842
KB (DiP) + LLM	0.793	0.954	0.865
KB (DiP + SP) + LLM	0.820	0.968	0.889
KB (DiP + SP + DyP) + LLM	0.963	0.968	0.965
Full LogPipe (continuous update)	**0.976**	**0.977**	**0.975**

Note - DiP: Discrete Patterns, SP: Semantic Patterns, DyP: Dynamic Patterns

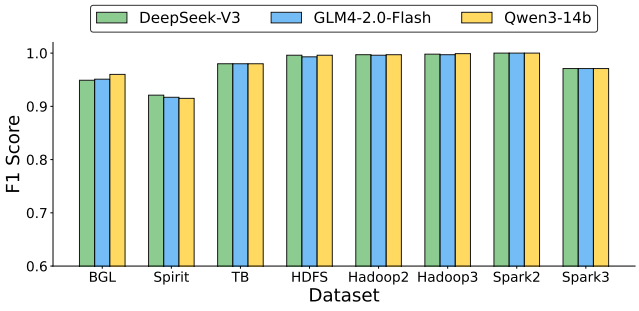


Figure 4: F1 Scores across LLMs: Performance Comparison

While the previous analysis demonstrates the effectiveness of the major modules of LogPipe, the choice of the LLM backbone represents another critical component that could affect overall model performance. To evaluate whether LogPipe's effectiveness is dependent on specific LLM architectures, we conduct a comprehensive evaluation using different LLM families, including DeepSeekV3 and Qwen3-14B, in addition to GLM4-2.0-Flash. As shown in Figure 4, the differences in anomaly detection performance across these LLMs are within 1%, indicating that LogPipe does not rely on a specific LLM.

Summary. Overall, our ablation study demonstrates that each major component of our framework is effective. Furthermore, LogPipe is robust across different LLMs.

5.3 RQ3: How does LogPipe perform in terms of explainability?

While anomaly detection has been greatly improved, determining the actual causes of detected anomalies remains a time-consuming task for engineers. Therefore, the explainability of anomalies has become increasingly important in practical applications. However,

verifying the correctness of anomaly explanations is challenging, as most publicly available anomaly detection datasets do not provide the exact causes of anomalies or the relevant log entries.

To address this, following the approach of PreLog [21], we formulate LogPipe's interpretability as the ability to identify failure-indicating log messages — that is, the specific log events that directly signal system faults. Notably, datasets such as BGL, Spirit, and Thunderbird provide status labels for individual log entries, which allow us to evaluate this log-level fault localization task. This task is performed only on pre-labeled anomalous sequences and does not involve anomaly detection. Accurate localization of these key events is a prerequisite for generating meaningful explanations. Importantly, the anomaly patterns stored in LogPipe's knowledge base can directly support this localization, providing structured guidance for identifying failure-indicating events. Once the indicative events are identified, we leverage an LLM to produce human-readable explanations.

Table 8: Comparison of Fault Localization Performance

Model	BGL		Spirit		TB	
	HR@1	HR@3	HR@1	HR@3	HR@1	HR@3
LR	0.300	0.350	0.400	0.420	0.830	0.970
DeepSeekV3	0.864	0.932	0.195	0.474	0.476	0.714
PreLog	0.870	0.961	0.901	0.982	0.900	1.000
LogPipe	**0.963**	**0.964**	**0.985**	**0.985**	**0.910**	**1.000**

Logistic Regression (LR) [3] enables fault localization by ranking the values of the learned weights for each log event in a single log sequence, reflecting their contribution to anomaly prediction. PreLog [21], leveraging a full attention mechanism, assigns attention scores to each log and localizes faults by ranking them. DeepSeekV3, with its strong semantic understanding, directly performs log-level fault localization in a zero-shot setting using prompts. In this study, we compare the fault localization performance of LogPipe with that of the three baselines. Following the setting adopted in PreLog [21], the fault localization is conducted on sequences of 20 logs each.

As shown in Table 8, our LogPipe achieved the best fault localization performance across all three datasets, significantly outperforming the data-driven Logistic Regression, the Transformer-based PreLog, and the powerful DeepSeekV3. This demonstrates the superb performance of our knowledge base. Furthermore, the knowledge base can help generate specific guidance that guides LLMs to produce detailed explanations.

Figure 5 illustrates the impact of the extra information on the LLM's judgment of log status and the generation of explanations. In the absence of extra knowledge guidance, the LLM misinterprets the log sequence in Input1, which may cause engineers to spend significant time investigating unrelated anomalies. However, when provided with specific guidance from a knowledge base, the LLM produces more specific explanations, offering substantial practical value to engineers. For example, in Input2, the LLM incorrectly flags the sequence as anomalous simply because the log entry E569 contains the word "invalid". This is because the LLM lacks a deep understanding of the system's operation and often judges anomalies solely based on keywords such as "error" and "invalid". However, these negative keywords do not necessarily indicate a genuine anomaly and could very well be normal logs generated during troubleshooting. In contrast, our knowledge base identifies E569 as a normal log template. With this contextual knowledge, the LLM gains a deeper understanding of system behavior, correctly recognizes the sequence as normal, and provides a reasonable explanation — E569 does not affect system functionality and is likely a benign log generated during certain operations or tests. Therefore, the extra information provided by the knowledge base helps reduce false positives from the LLM.

Table 9: The Results of Combining Anomaly Detection and Fault Localization

Model	BGL (100L)		Spirit (100L)		TB (200L)	
	HR@1	HR@3	HR@1	HR@3	HR@1	HR@3
LogPipe	0.820	0.820	0.898	0.900	0.910	1.000

While most existing anomaly detection methods overlook fault localization, our knowledge base inherently supports it by providing anomaly patterns based on log event IDs. Thus, we make a preliminary attempt to combine anomaly detection with log-level fault localization. Specifically, LogPipe first performs anomaly detection on log sequences (where the sequence lengths are set to 100 log entries for BGL and Spirit, and 200 for Thunderbird). It then performs fault localization on the identified anomalous log sequences. As shown in Table 9, LogPipe is capable of both detecting and localizing anomalies.

Summary. The experimental results demonstrate that LogPipe's knowledge base not only supports accurate anomaly detection and log-level fault localization but also enables the generation of more detailed explanations by guiding the LLM with structured, failure-relevant patterns.

6 Discussion

6.1 Why is LogPipe effective?

In terms of detection accuracy, LogPipe enriches LLMs with extra information about log sequences, allowing the LLM to develop a more comprehensive understanding of log messages. This reduces false positives and improves accuracy compared to approaches (such as RAGLog and LogPrompt) that rely solely on direct LLM classification without specific guidance. For efficiency, LogPipe caches "pattern–status–explanation" triplet. This design reduces the frequency of LLM queries, lowering inference costs and improving scalability in production environments. For explainability, LogPipe's explainability is specific because its knowledge base identifies fault-related log entries, as confirmed by improved fault localization performance. This validated knowledge then guides LLMs to generate specific and detailed anomaly explanations. Additionally, LogPipe requires a moderate number of labeled anomalies (around 2,000 abnormal log sequences in our experiments) for effective knowledge construction. In comparison, many supervised neural network-based methods [14, 47] demand large volumes of labeled data to learn generalized patterns, and their performance often degrades significantly in low-resource settings. This makes LogPipe more practical in real-world scenarios.

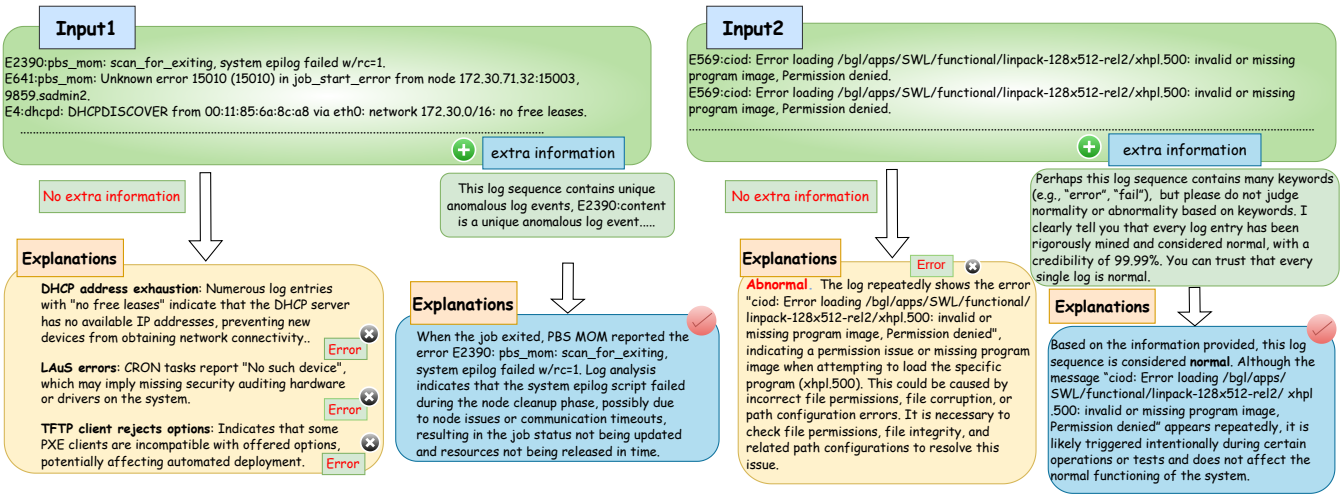


Figure 5: Extra information that guides LLM to explain

6.2 Threats to Validity

We have identified the following major threats to validity.

Subject Datasets. In this work, we evaluated our method using eight publicly available log datasets. Although these datasets are collected from real-world systems and contain millions of log entries, the number of systems covered is still limited and does not span all domains. Furthermore, some widely used datasets, such as HDFS and BGL, cover only a limited range of failure types, which may restrict a comprehensive assessment of the method's effectiveness across different fault scenarios [18]. To address this limitation, we additionally tested our approach on the recent datasets introduced in EvLog [16], which include a richer variety of failure types. In the future, we plan to evaluate our method on additional datasets collected from more diverse systems to further verify its robustness and generalizability.

Tool Comparison. In our evaluation, we compared LogPipe with several state-of-the-art baselines, including both traditional and LLM-based approaches. For a fair comparison, we adopted the official implementations from their publicly available replication packages and applied the same data preprocessing procedures. Hyperparameters and other settings (e.g., thresholds) were configured according to those reported in their original papers or official documentation. Still, the correctness of these implementations could be a threat.

Randomness. Bias from the randomness of LLMs may affect the evaluation. To mitigate this threat, we follow recent studies [17, 41] to reduce the randomness of the LLM by configuring it to generate consistent outputs for the same input text (i.e., initialize `temperature=0`).

Privacy. Using public LLM APIs may raise concerns about the potential leakage of sensitive information contained in logs. To mitigate this risk, we plan to deploy LLMs locally in future work to ensure the privacy of log data while maintaining the effectiveness of our approach.

Labelling Effort. Our experiments used 2,000 samples of anomalous log sequences. Although LogPipe requires less labeled data compared to many supervised methods, manual labeling efforts are still involved in our approach. In the future, we will explore unsupervised solutions to further reduce the dependence on labeled data.

7 Conclusion

In this paper, we propose LogPipe, a knowledge-augmented framework for log anomaly detection using large language models. By mining and organizing discriminative patterns into a structured knowledge base, LogPipe provides specific guidance to LLMs, improving detection accuracy, explanation quality, and cost-efficiency. Our extensive evaluation across eight public datasets demonstrates that LogPipe not only achieves state-of-the-art performance (0.975 average F1 score) but also reduces LLM token consumption by **5.711**× compared to direct LLM-based detection, requiring only **12.03%** of RAGLog and **18.96%** of LogPrompt. Moreover, LogPipe supports log-level fault localization and explanation generation, further enhancing its practical value. These results confirm the effectiveness of integrating structured knowledge with LLMs for robust and interpretable anomaly detection in real-world systems.

To support reproducibility and further research, we publicly release our experimental data and source code at: https://github.com/LogIntelligence/LogPipe.

Acknowledgments

This work is supported by National Natural Science Foundation of China (NSFC) grant 62572081. We also thank anonymous reviewers for their insightful and constructive comments, which significantly improve this paper.

References

[1] Crispin Almodovar, Fariza Sabrina, Sarvnaz Karimi, and Salahuddin Azad. 2024. LogFiT: Log Anomaly Detection Using Fine-Tuned Language Models. *IEEE Transactions on Network and Service Management* 21, 2 (2024), 1715–1723.
[2] Amazon Web Services. 2024. AWS Post-Event Summaries. https://aws.amazon.com/cn/premiumsupport/technology/pes/. [Online; accessed 30-April-2024].

[3] Peter Bodik, Moises Goldszmidt, Armando Fox, Dawn B. Woodard, and Hans Andersen. 2010. Fingerprinting the datacenter: automated classification of performance crises. In *Proceedings of the 2010 ACM SIGOPS Symposium on Operating Systems Principles (SOSP '10)*. ACM, 111–124.

[4] Mike Chen, Alice X Zheng, Jim Lloyd, Michael I Jordan, and Eric Brewer. 2004. Failure diagnosis using decision trees. In *International Conference on Autonomic Computing, 2004. Proceedings.* IEEE, 36–43.

[5] E. Chuah, S.-H. Kuo, P. Hiew, W.-C. Tjhi, G. Lee, J. Hammond, M. T. Michalewicz, T. Hung, and J. C. Browne. 2010. Diagnosing the root-causes of failures from cluster log files. In *2010 International Conference on High Performance Computing*. IEEE, 1–10.

[6] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (NAACL-HLT '19)*. ACL, 4171–4186.

[7] Min Du, Feifei Li, Guineng Zheng, and Vivek Srikumar. 2017. DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS '17)*. ACM, 1285–1298.

[8] Boris Galitsky, Anton Chernyavskiy, and Dmitry Ilvovsky. 2024. Truth-O-Meter: Handling Multiple Inconsistent Sources Repairing LLM Hallucinations. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '24)*. ACM, 2817–2821.

[9] Team GLM. 2024. ChatGLM: A Family of Large Language Models from GLM-130B to GLM-4 All Tools. *arXiv preprint arXiv:2406.12793* (2024). https://arxiv.org/abs/2406.12793

[10] Google LLC. 2024. Google Cloud Status Dashboard. https://status.cloud.google.com/summary. [Online; accessed 30-April-2024].

[11] Zihui Gu, Ju Fan, Nan Tang, Lei Cao, Bowen Jia, Sam Madden, and Xiaoyong Du. 2023. Few-shot Text-to-SQL Translation using Structure and Content Prompt Learning. *Proceedings of the ACM on Management of Data* 1, 2 (2023), 1–28.

[12] Wei Guan, Jian Cao, Shiyou Qian, Jianqi Gao, and Chun Ouyang. 2025. LogLLM: Log-based Anomaly Detection Using Large Language Models. *arXiv preprint arXiv:2411.08561* (2025). https://arxiv.org/abs/2411.08561

[13] Haryadi S. Gunawi, Riza O. Suminto, Russell Sears, Casey Golliher, Swaminathan Sundararaman, Xing Lin, Tim Emami, Weiguang Sheng, Nematollah Bidokhti, Caitie McCaffrey, et al. 2018. Fail-slow at scale: Evidence of hardware performance faults in large production systems. *ACM Transactions on Storage* 14, 3 (2018), 1–26.

[14] Hongcheng Guo, Jian Yang, Jiaheng Liu, Jiaqi Bai, Boyang Wang, Zhoujun Li, Tieqiao Zheng, Bo Zhang, Junran Peng, and Qi Tian. 2024. Logformer: A pre-train and tuning pipeline for log anomaly detection. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI '24)*, Vol. 38. AAAI Press, 135–143.

[15] Shilin He, Jieming Zhu, Pinjia He, and Michael R. Lyu. 2016. Experience Report: System Log Analysis for Anomaly Detection. In *2016 IEEE 27th International Symposium on Software Reliability Engineering (ISSRE '16)*. IEEE Computer Society, Los Alamitos, CA, USA, 207–218.

[16] Yintong Huo, Cheryl Lee, Yuxin Su, Shiwen Shan, Jinyang Liu, and Michael R. Lyu. 2023. EvLog: Identifying Anomalous Logs over Software Evolution. In *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE '23)*. IEEE, 391–402.

[17] Zhihan Jiang, Jinyang Liu, Zhuangbin Chen, Yichen Li, Junjie Huang, Yintong Huo, Pinjia He, Jiazhen Gu, and Michael R. Lyu. 2024. LILAC: Log Parsing using LLMs with Adaptive Parsing Cache. *arXiv preprint arXiv:2310.01796* (2024).

[18] Max Landauer, Florian Skopik, and Markus Wurzenberger. 2024. A Critical Review of Common Log Data Sets Used for Evaluation of Sequence-Based Anomaly Detection Techniques. *Proc. ACM Softw. Eng.* 1, FSE (jul 2024), 1354–1375.

[19] Van-Hoang Le and Hongyu Zhang. 2021. Log-based Anomaly Detection Without Log Parsing. In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE '21)*. 492–504.

[20] Van-Hoang Le and Hongyu Zhang. 2022. Log-based anomaly detection with deep learning: how far are we?. In *Proceedings of the 44th International Conference on Software Engineering (ICSE '22)*. 1356–1367.

[21] Van-Hoang Le and Hongyu Zhang. 2024. PreLog: A Pre-trained Model for Log Analytics. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–28.

[22] Tianle Li, Ge Zhang, Quy Duc Do, Xiang Yue, and Wenhu Chen. 2024. Long-context LLMs Struggle with Long In-context Learning. *arXiv preprint arXiv:2404.02060* (2024). https://arxiv.org/abs/2404.02060

[23] Xiaoyun Li, Pengfei Chen, Linxiao Jing, Zilong He, and Guangba Yu. 2020. Swisslog: Robust and unified deep learning based log anomaly detection for diverse faults. In *2020 IEEE 31st International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 92–103.

[24] Yinglung Liang, Yanyong Zhang, Hui Xiong, and Ramendra Sahoo. 2007. Failure prediction in IBM BlueGene/L event logs. In *Seventh IEEE International Conference on Data Mining (ICDM '07)*. IEEE, 583–588.

[25] Qingwei Lin, Hongyu Zhang, Jian-Guang Lou, Yu Zhang, and Xuewei Chen. 2016. Log Clustering Based Problem Identification for Online Service Systems. In *Proceedings of the 38th International Conference on Software Engineering Companion (ICSE '16)*. 102–111.

[26] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2023. Lost in the Middle: How Language Models Use Long Contexts. *arXiv preprint arXiv:2307.03172* (2023). https://arxiv.org/abs/2307.03172

[27] Peng Liu, Weize Yuan, Jia Fu, Zheng Jiang, Hideki Hayashi, and Graham Neubig. 2023. Pretrain, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM Computing Surveys (CSUR)* 1, 1 (2023), 1–35.

[28] Yilun Liu, Yuhe Ji, Shimin Tao, Minggui He, Weibin Meng, Shenglin Zhang, Yongqian Sun, Yuming Xie, Boxing Chen, and Hao Yang. 2025. LogLM: From Task-based to Instruction-based Automated Log Analysis. *arXiv preprint arXiv:2410.09352* (2025). https://arxiv.org/abs/2410.09352

[29] Yilun Liu, Shimin Tao, Weibin Meng, Feiyu Yao, Xiaofeng Zhao, and Hao Yang. 2024. LogPrompt: Prompt Engineering Towards Zero-Shot and Interpretable Log Analysis. In *2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion '24)*. 364–365.

[30] Daniel Loureiro, Francesco Barbieri, Leonardo Neves, Luis Espinosa Anke, and Jose Camacho-Collados. 2022. TimeLMs: Diachronic Language Models from Twitter. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics: System Demonstrations (ACL '22)*. ACL, 251–260.

[31] Siyang Lu, Xiang Wei, Yandong Li, and Liqiang Wang. 2018. Detecting anomaly in big data system logs using convolutional neural network. In *2018 DASC/PiCom/DataCom/CyberSciTech*. IEEE, 151–158.

[32] Lipeng Ma, Weidong Yang, Yixuan Li, Ben Fei, Mingjie Zhou, Shuhao Li, Sihang Jiang, Bo Xu, and Yanghua Xiao. 2025. AdaptiveLog: An Adaptive Log Analysis Framework with the Collaboration of Large and Small Language Model. *ACM Transactions on Software Engineering and Methodology* (jul 2025).

[33] Weibin Meng, Ying Liu, Yichen Zhu, Shenglin Zhang, Dan Pei, Yuqing Liu, Yihao Chen, Ruizhi Zhang, Shimin Tao, Pei Sun, et al. 2019. Loganomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs. In *Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI '19)*. IJCAI, 4739–4745.

[34] Microsoft Azure. 2024. Azure Status History. https://azure.status.microsoft/en-us/status/history/. [Online; accessed 30-April-2024].

[35] Jonathan Pan, Wong Swee Liang, and Yidi Yuan. 2024. RAGLog: Log Anomaly Detection using Retrieval Augmented Generation. In *2024 IEEE World Forum on Public Safety Technology (WFPST '24)*. IEEE, 169–174.

[36] Zhenwei Shao, Zhou Yu, Meng Wang, and Jun Yu. 2023. Prompting large language models with answer heuristics for knowledge-based visual question answering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR '23)*. IEEE, 14974–14983.

[37] Hudan Studiawan, Ferdous Sohel, and Christian Payne. 2021. Anomaly Detection in Operating System Logs with Deep Learning-Based Sentiment Analysis. *IEEE Transactions on Dependable and Secure Computing* 18, 5 (2021), 2136–2148.

[38] Chunqiu Steven Xia, Matteo Paltenghi, Jia Le Tian, Michael Pradel, and Lingming Zhang. 2024. Fuzz4all: Universal fuzzing with large language models. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE '24)*. 1–13.

[39] Yongzheng Xie, Hongyu Zhang, and Muhammad Ali Babar. 2022. LogGD: Detecting Anomalies from System Logs with Graph Neural Networks. In *2022 IEEE 22nd International Conference on Software Quality, Reliability and Security (QRS '22)*. IEEE, 299–310.

[40] Yongzheng Xie, Hongyu Zhang, and Muhammad Ali Babar. 2024. LogSD: Detecting Anomalies from System Logs through Self-Supervised Learning and Frequency-Based Masking. *Proceedings of the ACM on Software Engineering* FSE (July 2024), 1–23.

[41] Junjielong Xu, Ruichun Yang, Yintong Huo, Chengyu Zhang, and Pinjia He. 2024. DivLog: Log Parsing with Prompt Enhanced In-Context Learning. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE '24)*. 1–12.

[42] Wei Xu, Ling Huang, Armando Fox, David Patterson, and Michael I Jordan. 2009. Detecting large-scale system problems by mining console logs. In *Proceedings of the 22nd ACM SIGOPS Symposium on Operating Systems Principles (SOSP '09)*. ACM, 117–132.

[43] Lin Yang, Junjie Chen, Zan Wang, Weijing Wang, Jiajun Jiang, Xuyuan Dong, and Wenbin Zhang. 2021. Semi-supervised log-based anomaly detection via probabilistic label estimation. In *Proceedings of the 43rd International Conference on Software Engineering (ICSE)*. 1448–1460.

[44] Boxi Yu, Jiayi Yao, Qiuai Fu, Zhiqing Zhong, Haotian Xie, Yaoliang Wu, Yuchi Ma, and Pinjia He. 2024. Deep Learning or Classical Machine Learning? An Empirical Study on Log-Based Anomaly Detection. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE)*. 403–415.

[45] Hongyu Zhang. 2008. On the distribution of software faults. *IEEE Transactions on Software Engineering* 34, 2 (2008), 301–302.

[46] Wanhao Zhang, Qianli Zhang, Enyu Yu, Yuxiang Ren, Yeqing Meng, Mingxi Qiu, and Jilong Wang. 2024. Leveraging RAG-Enhanced Large Language Model

for Semi-Supervised Log Anomaly Detection. In *2024 IEEE 22nd International Symposium on Software Reliability Engineering (ISSRE '24)*. IEEE, 168–179.

[47] Xu Zhang, Yong Xu, Qingwei Lin, Bo Qiao, Hongyu Zhang, Yingnong Dang, Chunyu Xie, Xinsheng Yang, Qian Cheng, Ze Li, et al. 2019. Robust log-based anomaly detection on unstable log data. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 807–817.

[48] X. Zhang, Y. Xu, S. Qin, S. He, B. Qiao, Z. Li, H. Zhang, X. Li, Y. Dang, Q. Lin, et al. 2021. Onion: identifying incident-indicating logs for cloud systems. In *Proceedings of the ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '21)*. ACM, 1253–1263.

[49] N. Zhao, H. Wang, Z. Li, X. Peng, G. Wang, Z. Pan, Y. Wu, Z. Feng, X. Wen, W. Zhang, et al. 2021. An empirical investigation of practical log anomaly detection for online service systems. In *Proceedings of the ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '21)*. ACM, 1404–1415.

[50] Xiang Zhou, Xin Peng, Tao Xie, Jun Sun, Chao Ji, Dewei Liu, Qilin Xiang, and Chuan He. 2019. Latent error prediction and fault localization for microservice applications by learning from system trace logs. In *Proceedings of the 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '19)*. ACM, 683–694.

[51] Jieming Zhu, Shilin He, Pinjia He, Jinyang Liu, and Michael R. Lyu. 2023. Loghub: A Large Collection of System Log Datasets for AI-driven Log Analytics. In *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE '23)*. IEEE, 355–366.